

SIMATS ENGINEERING

ASSESSMENT TOOL 1 - Concept Mapping

COURSE NAME: Data Structures

COURSE CODE: CSA0308

Register Number	192511242
Name	Abishana S K

COURSE OUTCOME COVERED

CO1: *Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL2, BL3)*

Assessment Tool Weightage: 35%

Questions:2 EACH QUESTIONS CARRIES: 10 MARKS Total Marks:20 Duration: ONE HOUR

Q1 A smart healthcare system processes large volumes of patient data such as temperature readings, medical history, and diagnostic details. Construct a concept map linking the progression from Data → Data Object → Data Structure → Abstract Data Type (ADT) → Implementation in C. Identify relevant sub-concepts such as structures, pointers, and operations, and establish meaningful relationships among them. Organize the concept map hierarchically and justify how each level contributes to efficient data organization, processing, and improved system performance.

Q2 A smart city traffic management system handles data related to vehicle movement, road networks, and signal control. Construct a concept map connecting Primitive Data Structures → Non-Primitive Data Structures → Linear → Non-Linear Structures. Include examples such as arrays, linked lists, trees, and graphs, and illustrate their applications in traffic management. Explain the relationships between these structures and justify their classification based on functionality and efficiency.

Code of Conduct:

I Abishana S K (192511242) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

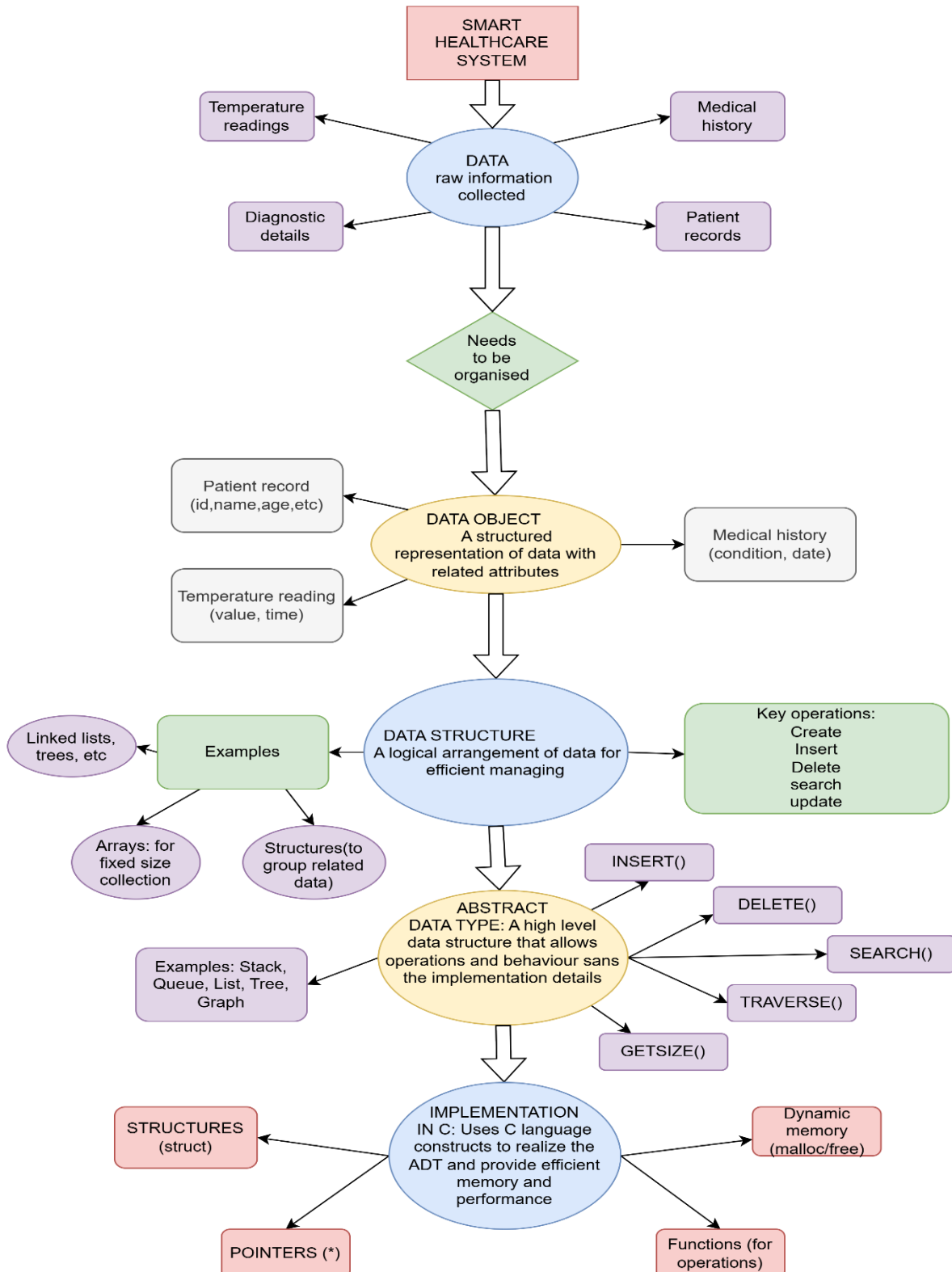
Signature of the student: Abishana S K

RUBRICS FOR CALCULATION:

Concept Mapping					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

ANSWERS:

1)



Smart Healthcare System:

The concept map represents the progression of patient information from raw Data to its practical Implementation in C using data structures and Abstract Data Types (ADTs).

Data refers to the raw facts collected by a smart healthcare system, such as patient temperature readings, medical history, diagnostic details, patient identification, and test results. These individual values provide the basic information required by the healthcare system.

The collected data can be represented as Data Objects. A data object is a specific entity or collection of values that can be stored and manipulated by a program. For example, a patient record may contain a patient ID, name, age, temperature, and diagnostic information. Data objects therefore provide a meaningful representation of individual healthcare entities.

These data objects are organized using appropriate Data Structures. A data structure defines how data is arranged and stored so that it can be accessed and manipulated efficiently. For example, arrays can store collections of similar patient readings, while linked structures can represent dynamically changing records. Structures in C can also group related attributes of a patient into one record. Pointers are important for accessing memory locations and for implementing dynamically allocated and linked data structures.

A Data Structure is associated with a set of operations used to manage the stored data. Important operations include creating or initializing data, inserting new records, deleting records, searching for information, updating values, and traversing the stored elements. The choice of an appropriate data structure affects the time and memory required for these operations.

The concept of an Abstract Data Type (ADT) provides a logical or abstract view of the data and the operations that can be performed on it, without specifying how those operations are implemented internally. Examples include List, Stack, Queue, Tree, and Graph ADTs. For example, a Queue ADT defines operations such as insertion and deletion according to queue behavior, while the underlying implementation may use an array or linked list.

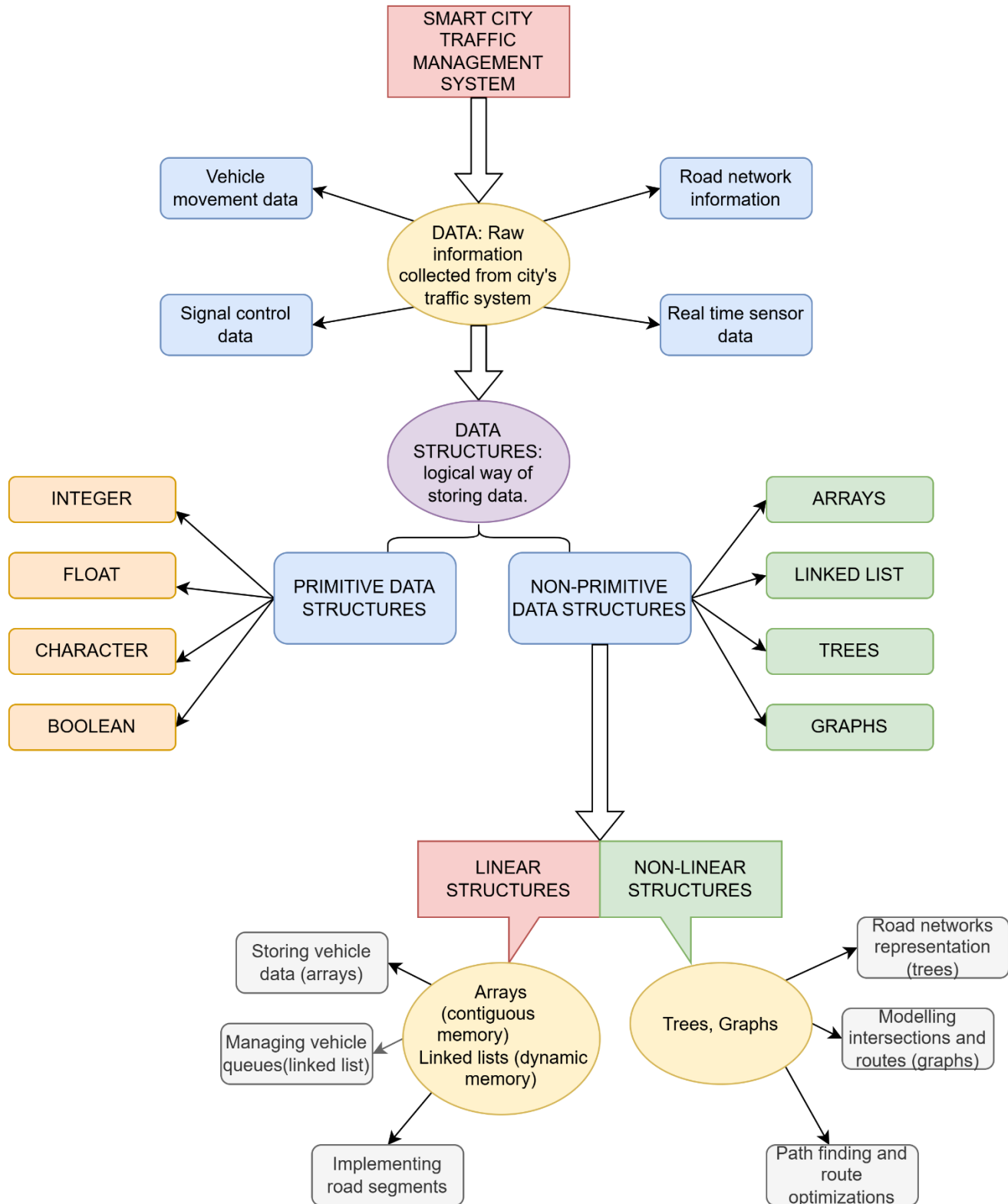
The ADT is finally realized through Implementation in C. C provides structures, pointers, functions, and dynamic memory allocation mechanisms that can be used to implement data structures and their operations. For example, a patient record can be represented using a 'struct', while pointers can be used to connect dynamically allocated records in a linked structure.

Thus, the relationship can be summarized as:

Data → Data Object → Data Structure → ADT → Implementation in C

Each level contributes to efficient healthcare data management. Raw data becomes meaningful data objects, data structures organize these objects for efficient storage and access, ADTs provide a clear interface for operations, and C provides the mechanisms for actual implementation. Selecting suitable data structures can reduce unnecessary processing and memory usage, improve searching and updating efficiency, and support faster processing of large volumes of patient data. Therefore, the complete progression contributes to improved system performance and effective organization of healthcare information.

2)



Smart City Traffic Management System: Classification of Data Structures

The concept map classifies data structures according to their nature and organization and connects them to their applications in a smart city traffic management system.

Data Structures provide systematic methods for storing, organizing, and processing traffic-related information such as vehicle movement, road networks, traffic signals, intersections, and sensor readings. Based on their nature, the question classifies them into Primitive Data Structures and Non-Primitive Data Structures.

Primitive Data Structures represent basic and fundamental data values. Examples include integer, float, character, and Boolean. In a traffic management system, these can represent individual values such as vehicle counts, vehicle speeds, signal timings, road identifiers, or status values. They are simple and are generally used as building blocks for more complex data organization.

Non-Primitive Data Structures are used to organize collections of data and represent more complex relationships. Examples relevant to the question include arrays, linked lists, trees, and graphs. Non-primitive structures can be further classified according to how their elements are organized into Linear and Non-Linear Structures.

Linear Data Structures organize elements in a sequential manner, where elements are arranged one after another. Arrays and linked lists are examples. An array can be used to store vehicle information or sensor readings in an organized sequence. A linked list can be useful when the number of elements changes dynamically, such as managing dynamically changing vehicle or traffic-event records.

Non-Linear Data Structures organize elements in hierarchical or network-based relationships rather than a single sequential order. Trees and graphs are examples. A tree can represent hierarchical relationships within a road or traffic-control system, while a graph is particularly useful for representing interconnected road networks, where intersections can be represented as vertices and roads as edges.

Graphs are especially useful in traffic management because they can represent routes and connections between different locations. They can support applications such as route finding, path analysis, and traffic-network modeling. Trees can represent hierarchical relationships, while arrays and linked lists provide suitable sequential or dynamic organization for different types of traffic data.

The relationship in the concept map can therefore be represented as:

Data Structures → Primitive / Non-Primitive → Linear / Non-Linear

Under the non-primitive category:

Linear → Arrays, Linked Lists

Non-Linear → Trees, Graphs

The classification is based on the functionality and organization of data. Primitive structures handle basic individual values, whereas non-primitive structures manage collections and relationships between data elements. Linear structures are suitable when data needs to be processed sequentially, while non-linear structures are suitable for hierarchical or interconnected relationships.

The choice of an appropriate structure also affects efficiency. Arrays provide organized access to collections of elements, linked lists allow dynamic organization, trees support hierarchical data representation, and graphs represent complex network relationships. In a smart traffic management system, selecting the appropriate structure allows vehicle data, road networks, and signal-related information to be organized and processed efficiently. This can contribute to faster data processing, better route management, reduced congestion, and improved traffic control.